

AI Vibration Intelligence Platform

Stage 1 — Progress Summary (Input → Process → Output)

Generated: June 10, 2026

This document summarizes work completed so far: Equipment & Sensor Master registration, database storage, sensor data upload, and five diagnostic vibration plots with formulas.

1. Platform Overview

Stack: React frontend (port 5173) · FastAPI backend (port 8000) · PostgreSQL (port 5433) · pgAdmin (port 5050)

Layer	Technology	Role
Frontend	React, TypeScript, Vite, Tailwind	UI forms, equipment list, vibration analysis plots
Backend	FastAPI, SQLAlchemy, Pydantic	REST APIs, validation, plot generation
Database	PostgreSQL 16 + Alembic	Persistent storage of equipment, sensors, uploads, config
Admin	pgAdmin	Browse tables, run SQL, verify stored data

2. Workflow A — Equipment & Sensor Master

Stage	Input	Internal Process	Output
INPUT	User fills 6-step form: • Plant / Area / Line • Machine name, type, criticality • Mechanical & rotating details • Operating conditions • Sensor config (type, location, orientation) • Optional equipment image	—	Form data in browser
PROCESS	—	1. POST /api/v1/equipment/ (JSON) 2. Pydantic validates request 3. CRUD inserts row in equipment_masters 4. Nested sensors → sensor_configurations 5. Image → POST /equipment/{id}/image → disk 6. Alembic manages schema migrations	—
OUTPUT	—	Data in PostgreSQL (viewable in pgAdmin)	• equipment_masters (1 row per machine) • sensor_configurations (1 row per sensor) • Image file path in DB; file on disk • UUID for each equipment & sensor

Database tables (Stage 1):

Table	Records	Key fields
equipment_masters	1 per machine	plant_name, machine_name, machine_type, 40+ technical columns
sensor_configurations	Many per equipment	sensor_type, mounting_location, orientation, sampling_rate
plot_configurations	1 per sensor	channel_count, active_channel, sampling_rate_hz, fft_lines, enabled_plots
sensor_data_uploads	1 per file upload	sensor_id, channel_count, parse_status, sample_count

3. Workflow B — Sensor Data Upload & Plots

Stage	Input	Internal Process	Output
INPUT	• sensor_id (UUID from equipment) • channel_count (e.g. 8) • CSV or PDF file (timestamp_, ch0...ch7) • Plot config (sampling rate, FFT lines, active channel)		Multipart upload + saved config
PROCESS	—	1. POST /api/v1/measurements/upload 2. pdf_parser extracts rows 3. Parsed JSON saved to disk 4. plot_configurations applied 5. signal_processing computes 5 plots 6. GET /uploads/{id}/plots returns x,y arrays	
OUTPUT	—	Plots rendered in Vibration Analysis UI	5 diagnostic plots for selected channel (ch0–ch7) • Zoom, pan, autoscale on each chart • Upload history per sensor

4. Key API Endpoints

API	Method	Purpose
/api/v1/equipment/	POST / GET / PATCH / DELETE	Equipment CRUD
/api/v1/equipment/{id}/sensors	POST / GET / PUT / DELETE	Sensor CRUD
/api/v1/equipment/{id}/image	POST / GET	Equipment photo upload
/api/v1/measurements/configure	POST / GET / PUT	Plot & acquisition settings per sensor
/api/v1/measurements/upload	POST	Upload CSV/PDF sensor data
/api/v1/measurements/uploads/{id}/plots	GET	Generate all 5 plots (?channel=0–7)
/health	GET	Backend health check

Data flow diagram: User → React UI → FastAPI → PostgreSQL (metadata) + File storage (images, parsed JSON) → Plot JSON → SVG charts

5. Plot Formulas (Backend: signal_processing.py)

Notation: $x[n]$ = sample amplitude, N = sample count, f_s = sampling_rate_hz, $t[n]$ = time in seconds.

5.1 Time Axis (shared)

If timestamp is epoch/batch ID: $t[n] = n / f_s$
Else: $t[n] = \text{timestamp}[n] - \text{timestamp}[0]$ (seconds or ms/1000)

5.2 Plot 1 — Time Waveform

$X = t[n]$, $Y = x[n]$ (raw amplitude vs time)

5.3 Plot 2 — Circular Time Waveform

$\theta[n] = 2\pi \cdot n / N$
 $X[n] = x[n] \cdot \cos(\theta[n])$, $Y[n] = x[n] \cdot \sin(\theta[n])$

5.4 Plot 3 — FFT Spectrum

Hanning window: $w[n] = 0.5 \cdot (1 - \cos(2\pi n / (N-1)))$
Windowed data: $x_w[n] = x[n] \cdot w[n]$
Magnitude: $M[k] = |\text{FFT}(x_w)[k]| \cdot 2/N$
Frequency: $f[k] = k \cdot f_s / N$

5.5 Plot 4 — Envelope Spectrum

Analytic signal: $z[n] = x[n] + j \cdot \text{Hilbert}(x[n])$
Envelope: $e[n] = |z[n]| - \text{mean}(|z|)$
Then FFT of $e[n]$ (same as Plot 3)

5.6 Plot 5 — Trend Plot

Split into 32 segments; per segment i :
 $\text{RMS}_i = \sqrt{(1/N_{\text{seg}}) \cdot \sum x[n]^2}$
 X = time at segment centre, $Y = \text{RMS}_i$

6. Frontend Pages Built

Page / Route	Status	Function
/equipment	Active	List, filter, delete equipment
/equipment/new & /equipment/:id/edit	Active	6-step Equipment Master wizard
/analysis	Active	Configure sensor, upload CSV/PDF, view 5 plots, switch ch0–ch7
/	Dashboard	Placeholder — upcoming analytics
/settings	Placeholder	Planned — acquisition config for edge script

7. What Is Complete vs Planned

Area	Status
Equipment & sensor registration → DB	✓ Complete
pgAdmin / PostgreSQL storage	✓ Complete
CSV/PDF upload & parsing	✓ Complete
5 diagnostic plots + formulas	✓ Complete
Plot zoom / pan / autoscale	✓ Complete
Configure API per sensor	✓ Complete
Edge UDP acquisition script integration	Planned — MAC mapping, own configure URL
Settings page for acquisition JSON	Planned
Dashboard live view from uploads	Planned
Replace external Sensovibe APIs	Planned